

Creeper — 设计报告

一份关于"如何把秘密藏进日常"的设计文档

版本 v1.0 | 2026-08-18

一、项目定位

Creeper 是一套**伪装成格式转换软件**的加密隐写工具：把任意文件加密后藏进 PNG 图片、MP3 与 WAV 音乐，载体照常观看、照常播放，只有知道暗号的人才能取回原件。

它不是保险箱——保险箱告诉别人"这里有值钱的东西"。

它是**日常本身**——秘密就摆在明处，只是没人知道那是秘密。

二、技术栈

层	选型	理由
语言	C++17 (GCC 15.2 / w64devkit)	单二进制、无运行时、性能无妥协
加密	Windows BCrypt (CNG) — AES-256-GCM + PBKDF2 60 万次	系统级实现，零第三方依赖，工业标准
图片	stb_image / stb_image_write (公域单头文件)	20KB 换整个 PNG 生态，无损编解码
音频	手写 MPEG 帧解析 (同步字/帧长/辅助位)	纯字节操作，只动帧头 3 个辅助位，无需解码器

UI	Dear ImGui + DirectX 11	现代暗色 UI，几十 KB 源码 换完整界面能力
中文字体	系统微软雅黑（运行时加载）	零打包体积，换电脑不缺字
便携性	静态链接，仅依赖系统 DLL	拷到任意 Win10/11 双击即用

选型哲学：能白嫖系统的不造轮子，能白嫖公版的不手写，手写的只留给必须手写的部分（MPEG 帧解析、纯软件密码学）。

三、原理

3.1 隐藏的数学：载体里的冗余

任何数字文件都有"人类感知不到的冗余"——这就是隐写的物理基础。

、

PNG：像素值 ± 1 （LSB matching）—— 颜色偏移 $1/255$ ，人眼不可察

嵌入只改 RGB 三通道各 1 bit，且只在与目标位不匹配时随机 ± 1 （不强制取值）

嵌入后对未承载像素做直方图配对补偿，把统计分布拉回原形（L1 距离降约 70 倍）

容量 = 宽 $\times$ 高 $\times 3 \div 8$ 字节（ $2560 \times 1600 \approx 1.46$ MB），默认只用 15%（ ≈ 230 KB）防统计检测

WAV：PCM 样本 LSB —— 每样本 depth 位（默认 1：只改最低位， ± 1 ，LSB matching）

样本值偏移 $1/32768$ （16-bit， $\approx -90\text{dBFS}$ ），不可闻；RIFF/fmt 区与高位零改动

容量 = 样本数 $\times$ depth $\div 8$ 字节（44.1kHz 立体声每分钟：depth=1 ≈ 661 KB，depth=2 ≈ 1.3 MB），

默认只用 15% (防统计检测), `--depth 2` / GUI「位深」高档可翻倍 (差 $\leq 3 \approx -78\text{dBFS}$, 仍不可闻)

—— 三种载体中容量最大, 大文件首选

MP3: MPEG 帧头辅助位 —— 每帧 private/copyright/original 3 个辅助位, 每帧 3 bit

音频数据区与 ID3 标签区逐字节零改动, 解码结果与原始文件完全相同

容量 = 帧数 $\times 3 \div 8$ 字节 (19,194 帧 ≈ 7.2 KB)

、

方案演进: 最初 PNG 用强制 LSB (位不匹配就改最低位), 会在直方图上留下 $H[2k] = H[2k+1]$ 的相邻 bin 阶跃; MP3 最初用 ID3v2 GEOB 帧, 会在标签区留下可见的 `creeper` 文件名与自定义数据——两者都是抽样检测一眼可辨的特征。现方案 (± 1 + 直方图补偿 / 帧头辅助位) 把统计痕迹压到与自然数据难以区分。**隐蔽性不是“看起来没变”, 而是“统计上没变”。**

3.2 加密的链条: 内容与访问分离

、

载荷文件

→ zlib 压缩 (最小化嵌入量, 减少统计暴露)

→ AES-256-GCM 加密 (密码 → PBKDF2 60 万次迭代 → 密钥)

→ 信封 (magic + 随机盐 + nonce + 密文)

→ 散布进载体 (PNG/WAV 用密码派生种子伪随机定位; MP3 帧序辅助位整体 XOR 密码派生 keystream)

、

设计要点: 内容安全靠 AES, 完整性靠 GCM 认证, 暴力破解靠 PBKDF2 迭代——每一层都是标准答案, 没有自创算法。自创密码学是业余的标志, Creeper 不做业余的事。

3.3 检测的存在性检测

载体是否含载荷，**不可在不解密的情况下判定**：隐写头无固定魔数（PNG/WAV/MP3 位流都以"文件名长度"开头，无任何固定特征；PNG/WAV 散布位置与 MP3 位流整体由密码派生，密码错连头结构都解析不出），唯一的判定路径是"用密码完整解析 → GCM 认证"（误报概率 2^{-128} ）。对外部校验者，任意文件都是"可能含也可能不含"——存在性问题被密码本身锁住，这正是"没有密码就不可见"的数学形式。

设计取舍：早期版本用固定魔数（CRPR/CRP）换取 $O(1)$ 检测，代价是**校验者按魔数模式匹配即可 $O(1)$ 发现载荷**；无魔数改造（NO-MAGIC-1）把检测代价从"一个字符串扫描"抬到"一次 60 万次 PBKDF2 + 全量解析"，且密码错误时一无所获。has 也因此从"检测命令"变为"密码验证命令"（has ）。

四、操作设计

4.1 三层伪装结构

第 1 层：程序本身 —— 一个正常的格式转换器（看起来是）

第 2 层：交互暗号 —— Ctrl+Shift+F 呼出元数据编辑窗（找得到入口的人才知道）

第 3 层：密码字段 —— "镜头格式" / "流派"（连密码框都不像密码框）

4.2 交互流程

正常使用（伪装层）：

添加文件 → 选输出格式 → 开始转换 → 完成

（图片是真转码 JPG↔PNG，经得起上手验证）

隐藏使用（真实层）：

加密：添加 2 个文件（宿主在前，载荷在后）

→ Ctrl+Shift+F → 镜头格式/流派 填密码 → 确定 → 开始转换

解密：添加 1 个文件（带载荷的宿主）

→ Ctrl+Shift+F → 填同一密码 → 确定 → 开始转换 → 得到原件

4.3 设计纪律

- **界面文案永不穿帮**：所有报错都伪装成转换器语义（"转换失败：密码错误或文件已损坏"），隐写概念零暴露
- **元数据窗不真改文件**：密码只进内存，EXIF/ID3 编辑是障眼，宿主文件零副作用
- **操作记忆成本低**：一个快捷键 + 一个字段，口诀一句话：Ctrl+Shift+F → 填密码 → 确定 → 转换

五、审美取向

5.1 伪装之美：像到极致就是隐身

Creeper 的审美第一原则——**不美，但像**。真实的格式转换软件是平庸的、工具化的、带一点年代感的：蓝色功能块、灰色按钮、朴素的进度条。所以界面刻意收敛，不做花哨设计，因为**太漂亮反而可疑**。

5.2 隐藏之美：临床般的干净

隐藏层的元数据窗是另一种审美：字段排列规整、预制数据真实可信（Canon EOS R6 Mark II、RF24-70mm F2.8、Neon Harbor 的《Midnight Drive》……），像一个认真生活的人随手拍的照片、常听的歌。

预制假数据不是乱填——**它本身就是伪装的一部分**。越像真的元数据，越没人多看它一眼。

5.3 代码之美：克制与诚实

- 头文件里写明格式规格，魔数、偏移、字节序清清楚楚
- 该用库的地方绝不手写，手写地方绝不偷懒（MPEG 帧长表、位级 BitStream、GF(2^128) 乘法逐位实现）
- 不追求“看起来厉害”的炫技代码，追求“三个月后还能读懂”的工程代码

六、优势

维度	Creeper	对比对象
载体无损	图片色差 $\leq 1/255$ 且直方图补偿；音频帧头外逐字节未动	多数工具藏完文件就“变脏”
零依赖	3 个 exe 共 6.3MB，仅依赖系统 DLL	同类工具常捆绑 Python/OpenSSL
伪装深度	三层伪装，界面经得起上手验证	普通隐写工具 UI 一眼暴露用途
密码学	标准算法 + 强 KDF + 认证加密，与 Python cryptography 交叉验证通过	自研算法的“玩具加密”
便携	文件夹拷走即用，说明书随行	需安装、需配置环境
反向工程成本	无固定魔数 + exe 字符串混淆，魔数、字段、流程全部需要逆向推断	工具特征公开即失效

七、愿景（使用场景）

7.1 现在能用

场景	玩法
----	----

云端隐私层	把敏感文档藏进日常照片/音频，丢进网盘/相册/音乐同步——照常展示，内容只有你知道
U 盘/移动盘	音乐文件夹里混进"带货"的 WAV/MP3，即使盘被借用也不会被察觉
社交传输	把加密文件藏进图片发出去，走正常聊天渠道，内容全程密文
个人档案	密码、密钥、证件扫描件，藏在家人也会翻的照片文件夹里
学习研究	隐写术、密码学、文件格式（PNG/RIFF/ID3v2）的完整动手实践

7.2 未来的方向

- **载体扩展**：BMP（无损 LSB 同套路）、JPEG 域（DCT 系数 + 直方图保持）。注：WAV 无损 LSB 载体已于 2026-08-19 落地（PCM 8/16-bit，默认每样本 1 bit，RIFF/fmt 零改动，容量 ≈ 661 KB/分钟，默认 15% 上限）；2026-08-19 深夜新增 **承载深度可配置**（`--depth 2` / GUI「位深」高 24bit $\rightarrow$ 每样本 2 bit，容量 $\times 2$ ，差 $\leq 3 \approx -78$ dBFS 仍不可闻，隐写头加 1B 深度字段，旧格式文件自动回退解析）
- **隐蔽性升级**：直方图补偿从"事后拉平"升级为"全程保持"、载荷分片抗检测（碎片散布多个宿主、单宿主截获无意义）。注：曾评估"纹理自适应嵌入"（按像素邻域纹理密度分配容量），但 ± 1 修改影响邻居统计达 2，任何内容阈值都存在模糊带导致掩码翻转、提取端无法重放序列——数学上不可行，已放弃；现以"纯密码派生散布 + 15% 填充率上限 + 直方图自对抗重嵌"覆盖同等防护意图
- **形态进化**：从"三个 exe"到"一个看起来人畜无害的相册/播放器"，伪装层融入日常软件形态
- **哲学终点**：当工具像到一定程度，它就消失了——使用者与工具之间只剩一层暗号

7.3 诚实的边界

Creeper 防"随便看看"，不防"专门来查"：高填充率下载体的统计偏移依然可测（卡方/RS 分析），宿主文件被转码压缩会毁掉数据，exe 逆向（XOR 混淆只是提高成本，不是不可逆）。 **± 1 嵌入 + 直方图配对补偿 + 自对抗重嵌已把痕迹压到很低（实**

测 L1 距离降约 70 倍、无相邻 bin 阶跃)，默认 15% 填充率上限把统计暴露限制在可忽略水平；散布位置与 MP3 位流整体由密码派生（无明文魔数、无明文 seed、无明文结构），"是否有载荷"本身不可判定——但原理上不存在完美隐写。它适合保护隐私，不适合对抗有明确怀疑对象的专业取证——这个边界写在说明书里，也写在这里，因为清醒是设计的一部分。

本设计文档与工具仅用于学习、隐私保护与合法用途。